实验二：软件设计与建模

项目：酒店预订系统

姓名：商世雄 学号：241820122 日期：2026-11-06

1 酒店预订系统的主要功能

本次实验选择酒店预订系统，根据实验一的需求完成UML和界面设计。系统主要供顾客预订酒店，以及酒店人员处理房量和订单。界面目前用于展示设计，业务后端在后续实验中实现。

实验一的requirements.md共有26条需求，其中22条已确认，包括18条功能需求、3条非功能需求和1条约束。其余4条涉及超时、停售修改和性能测试条件，本次在4.4节补充具体做法，不改动实验一的原始记录。

1.1 用户与权限

顾客：可以查询酒店、预订房型，查看和处理自己的订单。未登录的游客可以查询，但不能提交预订。

酒店工作人员：维护本酒店的房型、库存和价格，确认或拒绝预订，并办理入住和退房。

平台管理员：审核酒店、管理工作人员账号、处理异常订单，查看全平台的统计和操作日志。管理员处理订单时也需要遵守状态限制。

1.2 酒店查询与预订

组合查询：按城市、入住和离店日期、人数、房型和价格筛选。只有整个住宿区间都有足够房量，才显示为可预订。

提交预订：顾客选择房型，填写联系人、间数和人数，确认逐夜价格与总价后提交。提交成功后生成待确认订单，并预留各晚房量。

价格保存：订单保留下单时的逐夜成交价，酒店后来改价，不影响已经生成的订单。

1.3 房型和每日房量管理

房型资料：酒店人员设置房型名称、可住人数和房间总数。

每日房量：分别维护每天的供给、已占用量、价格及停售状态。减少供给时，不能少于已经占用的数量。

1.4 订单处理

确认与拒绝：酒店处理待确认订单。确认后保留原预留，拒绝后归还相应房量。

修改与取消：待确认订单可以改期、换房型或调整间数；已确认订单只能改联系人。取消时检查订单状态和截止时间。

入住与退房：由本酒店工作人员核对订单和顾客身份后办理，不保存身份证照片。重复操作不能再次改变状态或库存。

异常处理：对超时、未入住、临时停业等情况分别处理，保留原订单和原因记录。

1.5 统计和操作记录

酒店人员可以查看本酒店的有效预订数、间夜数和操作日志，管理员可以查看全平台。日志记录操作者、时间、对象、前后状态和处理结果，电话等信息脱敏显示。

住宿费用在线下结算，系统不接入支付、退款、评价、优惠券、地图、会员积分或第三方渠道。房量只按房型管理，不分配具体房间，也不处理清洁维修、提前入住、延迟退房和续住。通知保存为平台待办，由工作人员线下联系。

2 软件架构及模块之间的关系

系统分为展示层、应用层、领域层和持久化层，业务模块放在同一个应用中，使用同一个数据库。预订时要同时生成订单、占用多个日期的库存，只要其中一步失败就要全部撤销。因此，本项目没有采用微服务，而是把这些操作放在同一个数据库事务中处理。

模块	负责的功能	对应需求
M01 账号与权限	顾客注册、各类用户登录、账号启停；检查操作者是否有权访问目标订单或酒店	REQ-001、023
M02 酒店与房型	酒店审核启停、房型资料、每日房量、价格和停售设置	REQ-002、003
M03 查询与报价	按城市、日期、人数等条件查房，计算逐夜价格和总价；提交时重新检查	REQ-004、007
M04 预订处理	创建订单、确认拒绝、修改取消、入住退房；处理重复请求和旧页面操作	REQ-005—016、020
M05 库存与价格	预留和归还各晚房量，计算改单后的库存变化，保存成交价	REQ-003、006、007、011
M06 异常处理	检查超时、未入住和停业影响，调用预订服务完成相应处理	REQ-010、015、017
M07 统计与日志	汇总有效预订和间夜数，按权限查询订单历史和操作日志	REQ-018、019
M08 数据保存与备份	保存业务数据，维护关联和唯一约束，执行备份、恢复及通知重试	REQ-020、024

页面只收集输入和展示结果，具体业务交给应用服务。应用服务先检查权限和订单状态，再调用库存服务、Repository等完成处理。这样，顾客页面、酒店页面和定时任务不会各写一套扣房规则。统计模块只读取数据，不修改订单和库存。

2.1 顾客端与查询、预订模块

逻辑关系：顾客通过界面输入条件，查询模块返回可订房型和报价；提交预订后，由预订模块再次检查价格和库存，生成订单。

设计原因：查到有房和提交成功之间可能有时间差，需要在提交时重新检查。界面负责收集信息，库存处理放在服务端。

2.2 预订模块与库存模块

逻辑关系：创建、修改和取消订单时，预订模块调用库存模块，处理受影响的全部住宿夜。

设计原因：一笔订单可能住多晚，订单和各晚房量需要同时更新。将库存操作集中处理，可以避免某个页面漏扣或重复归还。

2.3 异常处理与预订模块

逻辑关系：超时任务、未入住任务和管理员异常操作，都通过预订服务改变订单状态。

设计原因：自动任务与页面操作使用同一套检查，已取消或已入住的订单就不会被其他处理流程随意覆盖。

2.4 统计、日志与业务数据

逻辑关系：统计模块读取订单数据，日志模块记录和查询关键操作，两者都按用户身份限制范围。

设计原因：酒店人员只需要处理本酒店业务，顾客只应看到自己的订单；统计查询也不应改变原有订单和房量。

UML部分共9张图，涉及7种类型。组件图、两张类图和部署图说明系统结构；用例图、状态图和活动图说明业务行为；两张时序图分别说明创建预订和修改预订时的调用顺序。各图均用PlantUML绘制，保留源文件和导出图片。

3 类与数据设计

3.1 User与Booking

逻辑关系：一个顾客可以有多笔订单，每笔订单只属于一个顾客。三类用户共用用户表，通过role区分，工作人员另绑定一个hotelId。

设计原因：订单需要能查到预订人，也要据此限制访问范围，不能让顾客查看别人的订单。

3.2 Hotel、RoomType与RoomDay

逻辑关系：一家酒店有多种房型，每种房型按日期保存RoomDay记录，记录当天的供给、占用和价格。

设计原因：同一种房型在不同日期可能有不同价格和余量，不能只在房型上保存一个总库存。订单中的房型还必须属于该订单的酒店。

3.3 Booking与BookingNight

逻辑关系：Booking表示订单，BookingNight保存每个住宿夜的成交价和占用情况。一笔连住三晚的订单对应三条逐夜记录。

设计原因：跨日库存和价格都需要逐晚计算。逐夜成交价与酒店当前价格分开保存，酒店调价时不会改变旧订单。

3.4 BookingService与其他服务

逻辑关系：BookingService组织订单操作，调用InventoryService计算房量变化，并通过AuthorizationPolicy和TransitionPolicy检查权限、状态及操作时限。

设计原因：界面和定时任务都从同一服务处理订单，便于统一控制一次操作的成功、失败和数据回滚。

3.5 数据保存

主要数据及其限制如下。

数据表	主要内容和限制
app_user	id为主键，账号不能重复；保存密码哈希、角色、酒店编号和启用状态。工作人员只能绑定一个酒店
hotel	保存名称、城市、地址、联系方式、审核及营业状态；time_zone使用IANA时区名，如Asia/Shanghai；按城市和状态建立索引
room_type	id为主键，hotel_id关联酒店并建立索引；capacity（可住人数）和total_rooms（房间总数）均为正整数
room_day	以房型编号和住宿日期为联合主键；保存供给、占用、价格、停售标记和版本。已占用量不能为负或超过供给，供给不能超过房型总房数
booking	保存顾客、酒店、房型、入住离店日期、间数、人数、联系人、状态、总价和版本；另记确认计时起点及截止时刻。离店日期须晚于入住日期，间数和人数均为正整数
booking_night	以订单编号和住宿日期为联合主键，关联对应的房型日历，保存成交单价、间数和占用状态；同一订单同一晚只有一条记录
request_record	操作者、操作类型和请求键的组合不能重复；保存请求内容哈希及处理结果，供重复提交时返回原结果
audit_log	保存操作者、对象编号、酒店、动作、前后状态、结果、原因和时间，提供只读查询
notification_task	保存关联订单、通知内容、处理状态和重试次数，通知失败后可以重试

订单查询主要按顾客、酒店、状态和入住日期进行，因此计划建立(customer_id,created_at)、(hotel_id,status,check_in)索引。超时任务使用(status,expires_at)索引。日志按酒店或操作者与时间组合建立索引。查询结果分页返回，实际效果在后续测试中检查。

取消订单、停业或停用账号都不删除历史订单。订单修改后保留新的逐夜明细，旧日期、价格及联系人变化记入日志。日志中的电话先脱敏，不保存密码和整份请求。日志保留180天，由系统定期清理，管理页面不提供修改和删除功能。

4 主要业务处理

4.1 跨日库存与价格

库存按住宿夜计算，入住日计入，离店日不计入。例如11月10日入住、12日离店，只占10日和11日的房量；预订两间，则这两晚各占两间。当天还能出售多少间，等于当天供给减去仍应占用的房量。

预订提交成功后，订单进入待确认，同时预留全部住宿夜的房量。酒店确认和办理入住时不再扣一次库存。如果任一晚无房、停售、没有日历数据，或处理中写入失败，整笔预订都不成功，已做的修改全部撤销。查询和提交之间可能有人订走房间，所以提交时还要重新检查。

订单总价是各晚单价相加后再乘房间数。例如两晚分别为299元和359元，订一间共658元。提交时带上顾客已经接受的报价版本；若当前价格变了，先展示新价格，让顾客重新确认。生成订单后保存成交价，后续酒店调价不影响这笔订单。

4.2 并发预订和重复提交

创建预订时，先登记请求键，检查酒店营业状态和房型，再锁定所需日期的库存，核对价格和余量。检查通过后，写入预留、订单、逐夜价格、处理结果、操作日志和通知待办。这些写入由同一个UnitOfWork管理，要么一起提交，要么全部撤销。失败原因在事务回滚后另行记录，避免日志也跟着被撤销。

身份校验和查询已有结果可以先做，但登记新请求键必须放在事务中，并由数据库唯一约束防止重复。如果两个相同请求同时到达，后一个等待前一个完成，再读取结果；前一个回滚时则允许重试。网络超时不能直接当作预订失败，顾客可以用原请求键查到结果，避免另建一笔订单。

两人同时订最后一间时，不能都根据页面上的旧余量下单。处理程序按房型编号、日期的固定顺序锁住库存记录，读取最新余量，确认变更后不会超过供给才更新。

下单时对酒店和房型使用共享锁，酒店停业和房型维护使用与其冲突的锁，避免检查完营业状态后酒店刚好停业。改单先锁订单，再依次锁酒店、房型和日历。停业操作只更新酒店状态，之后逐单取消时分别开事务，不在持有酒店锁的同时反向锁住全部订单。

请求键与操作者、操作类型一起判断是否重复。同键、同内容返回原结果；同键但内容不同，返回409和REQUEST_KEY_CONFLICT，原结果不变。顾客确实要再订一笔时，使用新的请求键。“无房”这类已处理完的结果可以保存，临时服务器故障则允许原键重试。

修改、确认和取消都带上页面读取到的订单版本expectedVersion。实际处理时版本或状态已变，就返回409并要求刷新。权限也由服务端重新检查，不能只靠页面隐藏按钮。如果酒店先确认成功，顾客刷新后仍可在允许时限内取消；如果顾客先取消成功，酒店就不能再确认。

4.3 修改、取消和库存释放

“待确认”表示顾客已提交订单，酒店还没有确认。这时可以修改日期、房型、间数和联系人。修改时锁定新旧日期涉及的库存，逐日计算新占用量减去原占用量。重叠日期的房量没有变化，就保留原来的占用，不再扣减。

例如原订单在10日和11日各占一间，改成只住10日，就保留10日的占用，归还11日的一间。如果想延住到12日，但12日没有房，则整次修改失败，原订单、价格和预留都不变。保存前要让顾客接受

新价格；保存成功后重新计算30分钟的酒店确认时限。

酒店已经确认的订单只能改联系人，不重新计时。日期、房型或间数有变，需要按规则取消后重订。改单限于原酒店，换酒店也需要重新预订。

顾客可以取消待确认订单。已确认但尚未入住的订单，只能在入住日酒店当地18:00前取消，18:00整及以后不再允许。取消或拒绝后归还尚未消费的预留，重复操作不再次归还。正常退房不恢复已经住过的房量。

到了酒店当地入住日次日00:00，已确认但仍未入住的订单标为“未入住”。一晚订单没有后续房量可归还；三晚订单只归还后两晚。BookingNight用HELD、CONSUMED、RELEASED区分预留、已消费和已归还，过去的住宿夜不重新出售。

4.4 停业、超时及补充规则

酒店临时停业后，先停止接受新预订。管理员逐笔处理待确认和已确认订单，填写原因后取消，并在同一事务中归还预留。通知先记为待办，失败后重试。已入住订单不直接改成已退房，由平台与酒店联系住客处理。恢复营业需管理员手动启用。

房型停售与酒店停业分开处理：单纯停售不取消原订单，但不能在停售日期增加占用。具体规定如下。

编号	处理事项	采用的规则
P01 / REQ-010	30分钟截止	到期即不能确认或修改，不因后台任务延迟而延长。到期前已合法确认的订单，不再进入超时处理
P02 / REQ-017	停售和停业时的修改	停售日期可保持或减少原有占用，不能增加；其他日期仍检查可售和余量。酒店停业后不允许改日期、房型和间数。联系人修改及取消按原状态规则处理
P03 / REQ-021、022	性能测试条件	使用第6节的固定环境，查询和预订分别做20并发、100次测试，失败及超时均计入统计

订单创建或修改成功时，将本次计时起点加30分钟，存入expiresAt。处理确认时，锁定订单后读取服务器时间，必须早于expiresAt，恰好到期也拒绝。后台每5秒检查一次到期订单；确认、修改等写接口也做这项检查，发现已到期就先转为已超时、归还一次库存，再返回超时提示。

每家酒店保存IANA时区名，例如Asia/Shanghai。入住和离店按当地日期记录，创建、截止和日志时间存为UTC时间戳，显示时再换算。取消的18:00和未入住判定的00:00都按酒店当地时间计算。

查询时不填日期，可以浏览酒店和房型资料，但不显示整段总价和是否可订，价格筛选也不可用。只填了入住或离店中的一个日期，就提示补齐。

统计按计划入住日计入，开始日和结束日都包含。例如查询11月1日至30日，1日和30日入住的有效订单都算在内，不把跨区间订单拆成每晚统计。只统计已确认、已入住和已退房订单。

易用性测试安排5名首次使用者，每人分别用电脑和手机完成一次查询及预订。两端分别计时，从填写查询条件到看到预订成功结果为止，出错和重试也算在内。测试人员不代操作，至少4人两端都在4分钟内完成，并记录设备、浏览器版本和用时。

5 各功能模块的界面及功能说明

界面采用左侧导航、右侧内容的布局，顾客、酒店人员和管理员分别使用自己的页面。状态同时用颜色和文字表示，避免只看颜色判断。各界面见后附UI图。

界面源文件为diagrams.net的.drawio格式，文字和控件可以分别编辑，共导出16张PNG。另外保留了可点击的HTML原型，演示页面跳转和操作反馈，不连接真实后端。

原型中可以查看日期填错、提交前改价、改单无房、重复取消、库存调低和酒店停业后的提示。实际的登录鉴权、数据库锁、请求去重和定时任务仍要在后续实验实现。

5.1 登录与注册（UI-01）

账号输入：填写账号、密码并选择登录身份。顾客可以注册，工作人员账号由管理员创建。

登录后进入相应页面。实际系统仍由服务端检查身份，不能只凭页面选择获得权限。

5.2 酒店查询和房型详情（UI-02、UI-03）

条件筛选：填写城市、入住和离店日期、人数、间数及价格范围，选择需要的房型。

结果显示：列表显示符合条件的酒店和房型。点击查看后，可以看到各晚价格、余量和整段总价，再进入预订页面。

输入的日期不合法时，提示修改；没有结果时显示暂无符合条件的可订房型。

5.3 填写预订（UI-04）

入住信息：填写联系人、电话、房间数和入住人数。

价格明细：右侧显示逐夜单价和本次总价。顾客确认后提交；若提交前价格发生变化，需要再次确认。

5.4 我的订单和订单详情（UI-05、UI-07）

订单列表：按状态查看自己的订单，点击进入详情。

订单详情：显示酒店、房型、日期、联系人、逐夜成交价和操作历史。根据当前状态提供修改或取消入口，电话脱敏显示。

5.5 修改预订（UI-06）

方案对照：左侧显示原订单，右侧填写新的日期、房型和间数，方便对照变化。

保存修改：顾客接受新报价后保存。新日期无房或停售日期需要增加占用时，修改失败，原订单不变。缩短住宿等不增加停售日期占用的修改，仍按规则检查。

5.6 房型和每日库存管理（UI-08、UI-09）

房型资料：酒店人员填写名称、可住人数和房间总数。

日历管理：按日期修改供给、价格和停售状态。表格把供给总量、已占用和剩余分开列出；修改后的总量不足以覆盖已有占用时，不允许保存。

5.7 确认和拒绝预订（UI-10）

酒店人员查看本酒店的待确认订单，选择确认或填写原因后拒绝。确认不重复扣房，拒绝后归还预留。

页面显示确认截止时间。恰好到期或已经超时的订单不能再确认，需要刷新状态。

5.8 入住和退房（UI-11）

订单查询：通过订单号或联系方式查找本酒店订单。

现场办理：核对顾客和订单信息，记录核验结果后办理入住。已入住订单可以正常退房；重复点击不产生第二次操作，不保存证件照片。

5.9 酒店审核与异常处理（UI-12）

酒店管理：管理员查看审核状态，审核酒店或设置营业、停业。

停业处理：先显示受影响的待确认、已确认和已入住订单。管理员填写原因，逐笔处理未入住订单；已经入住的订单转为人工联系处理，不直接变成已退房。

5.10 工作人员账号（UI-13）

管理员填写账号、姓名和所属酒店，创建工作人员账号。每个账号只绑定一家酒店，也可以停用，原来的操作记录继续保留。

5.11 统计查询（UI-14）

选择计划入住日期和房型，查看有效预订数与间夜数。统计包含开始日和结束日，工作人员只查本酒店，管理员可以查全平台。

界面中的示例为3单9间夜，分别来自两间三晚、一间一晚和一间两晚。

5.12 操作日志（UI-15）

按操作者、时间和操作类型组合查询。每条记录显示对象、动作、前后状态、结果和原因，只能查看，不能在页面中改写。

5.13 手机端订单详情（UI-16）

手机页面保留酒店、房型、住宿日期、联系人和价格明细，修改与取消按钮放在下方。使用的状态和时限规则与电脑端相同。

6 非功能设计

检查内容	设计做法	验证要求
权限和密码	服务端角色+归属校验、受限数据库账号、参数化SQL、加盐单向哈希、日志脱敏；Cookie会话采用HttpOnly/Secure/SameSite并校验CSRF	三角色至少30组非法请求全部拒绝且业务数据不变；直接请求不能绕过页面限制
查询性能	city/status、房型日历联合索引、分页、按区间聚合最小余量，不缓存可售库存作为最终提交依据	使用本节统一测试基线，20并发100次，至少95次在2秒内正确返回
预订性能与正确性	短事务、固定锁序、最后一间锁后重查、请求唯一约束；通知在提交后处理	同环境20并发100次，至少95次3秒内明确结果，全部不得超卖；同时包含充足库存成功场景
可靠性	原子事务、幂等重试、审计与通知待办、每日备份和隔离恢复演练	每24小时备份；2小时内恢复至最近成功备份，RPO≤24小时；正常事务不可部分提交
易用性	字段级错误提示、总价口径说明、状态相关操作、手机详情适配、清晰的破坏性操作确认	5人分别测试桌面和手机，至少4人两端均在4分钟内独立完成；计时包含出错和重试

后续性能测试按下表准备环境和数据。

项目	测试配置
服务端	8核逻辑CPU、16GB内存、100GB SSD，Ubuntu 22.04 LTS 64位；一个应用实例与一个数据库实例运行在同一测试主机，不部署其他高负载服务
软件	OpenJDK 17、Spring Boot 3.3、PostgreSQL 16、Nginx 1.24；正式测试时记录具体补丁版本和配置，不在测试中途更换
压测端与网络	独立的4核、8GB主机，JMeter 5.6.3；千兆局域网，往返延迟不超过10毫秒；客户端从发出请求至完整收到响应计时
数据规模	10000家酒店、每家5种房型，共50000房型；100000条订单；每种房型准备连续30个住宿日，共1500000条日历记录，库存与订单预留一致
查询请求	20名并发用户，每人5次，共100次；1、2、3、7晚各25次，条件覆盖城市、人数、房型、价格和无结果场景。至少95次在2秒内正确返回，错误和超时均计不达标
预订请求	先做4轮20并发的充足库存预订，共80次；再以20个不同请求键竞争同房型同区间最后一间，共100次。前80次应成功，最后20次仅1次成功、19次明确返回无房；至少95次3秒内返回业务结果，不得超卖
执行方式	查询与预订分开测试，每项开始前恢复同一初始数据。记录每次条件、耗时、结果和库存；不把失败剔出分母，不用全部拒单替代成功预订场景

本次已检查图形渲染、原型演示和文件完整性。表中的性能、安全、恢复和首次使用测试，要等业务系统实现后再做。

7 辅助工具的使用

本次通过OpenCode使用GPT-6.1 Sol（openai/gpt-6.1-sol）做AI辅助设计。输入材料是实验一需求和实验二任务要求，主要用来生成UML草稿、界面原型及检查脚本。

7.1 功能整理

把实验一需求交给模型，先按用户和业务流程列出主要功能，再对应到账号、酒店、预订、库存等模块。这个步骤用来整理材料，没有加入支付、评价等原需求之外的功能。

7.2 UML绘制

模型生成PlantUML草稿后，先检查能否渲染，再看类之间的关系、订单状态和调用顺序。状态图曾因中文名称未加引号而报错，修正后重新生成。服务类图第一次过宽，放入报告后字太小，后来调整了布局，保留原有调用关系。

7.3 界面设计和修改

模型参与了界面控件、可编辑源文件和HTML原型的生成。之后检查表格文字、页面跳转和错误提示，补充到期不能确认、停售不能增间等演示。报告的版式和表达也按阅读反馈调整过数次，具体过程保存在审查与版本记录中。

7.4 检查与版本保存

通过脚本检查图形渲染、文件数量和原型操作。最初检查7组操作，补充边界情况后共12组演示通过，没有出现JavaScript异常。这些是原型和文件检查，不是后端系统的性能或并发测试。

初稿及后续修改分别提交到Git。初稿标签为lab2-ai-draft，第一轮技术检查的标签为lab2-technical-check。制作目录中的审查与版本记录保留了具体过程，业务设计的本人核对记录尚待补充。

业务部分还要重点核对改单时的重叠日期、酒店停业与下单同时发生的情况，以及各角色能操作哪些订单。

已完成的技术修改如下。

图片处理：将过宽的类图从2545像素调整到约1400像素，报告图页按方向和大小排版；较长的界面文字按列宽调整，避免挤到下一列。

规则补充：超时按严格截止处理，停售日期不能增加占用，并明确时区、统计日期和测试条件。图、界面及文字说明一同更新。

报告整理：改为普通白底宋体，缩小标题，删去重复说明。提交包只保留规定的35个文件，并与提交目录逐字节核对。

7.5 工具的优点和不足

优点：可以较快地整理需求和生成PlantUML代码，修改关系或文字后也方便重新出图。对于表格、界面控件和重复的文件检查，能减少一些制作工作。

不足：生成的图不一定适合直接放入报告，代码也可能有语法错误。对于订单时限、库存归还和权限等问题，仍需按需求判断，不能只因为图能生成就认为设计正确。

后续仍按整理需求、生成草稿、检查修改和保存版本的顺序使用。语法、文件数量和部分演示操作可以交给脚本，但业务取舍还需要人工判断，不能把检查与校对完全等同于完成设计。

8 需求对应关系

需求	对应设计	图 / UI
REQ-001	M01，角色与对象归属	用例/服务类；UI-01、13
REQ-002	M02，酒店审核营业状态	组件/领域类；UI-03、12
REQ-003	M02、05，房型与日历供给	领域类；UI-08、09
REQ-004	M03，组合条件及全程库存	活动；UI-02、03
REQ-005	M04，原子建单	创建时序；UI-04
REQ-006	M05，排序锁定与净变化	领域类/两张时序；UI-06、09
REQ-007	M03、05，逐夜报价与快照	创建时序；UI-03、04
REQ-008	M04，RequestRecord唯一键	服务类/创建时序；UI-04、07
REQ-009	M04，确认拒绝	状态；UI-10
REQ-010	M06，严格截止及写接口补做超时处理	状态/服务类；UI-05、10
REQ-011	M04、05，改单净变化	修改时序；UI-06
REQ-012	M04，取消状态及截止	状态；UI-07
REQ-013	M04，现场核验入住	状态/用例；UI-11
REQ-014	M04，正常退房	状态；UI-11
REQ-015	M06，未入住只放未来夜	状态/服务类；UI-11
REQ-016	M04，策略与版本防护	状态/两张时序；UI-05—11
REQ-017	M06，停业逐单处置；停售日期禁止新增占用	组件/状态；UI-09、12
REQ-018	M07，按计划入住日统计	组件；UI-14
REQ-019	M07，审计与授权查询	领域类/部署；UI-07、15
REQ-020	M08，外键唯一与事务	领域类/服务类；UI-13
REQ-021	查询索引及固定测试基线	部署；UI-02
REQ-022	短事务、充分库存及最后一间竞争压测	创建时序/部署；UI-04
REQ-023	授权、哈希与脱敏	服务类/部署；UI-01、15
REQ-024	原子性、备份和恢复	两张时序/部署；报告第6节
REQ-025	首次使用任务和跨端适配	UI-02、04、16；报告第6节
REQ-026	范围约束	报告第1节；所有图/UI均不含支付退款

9 实验小结

通过本次实验，对软件设计与建模有了进一步了解，也熟悉了几种常用UML图的用途。把需求分析中的功能落实到模块和界面后，对系统的整体结构有了更清楚的认识。设计时不仅要考虑功能是否完整，也要注意各部分之间的配合和界面的使用方式。

目前的设计还有不够细致的地方，后续实现时需要结合实际情况继续调整。希望通过后面的实验，把设计中的主要功能逐步实现，并在测试中发现问题、加以完善。

后附UML和界面图，提交目录中保存原尺寸图片及PlantUML源文件。

图1 组件图

结构性图。逻辑关系：三个客户端调用各业务模块，再访问数据库。设计原因：订单和库存需要一起更新，放在同一应用和事务中处理。

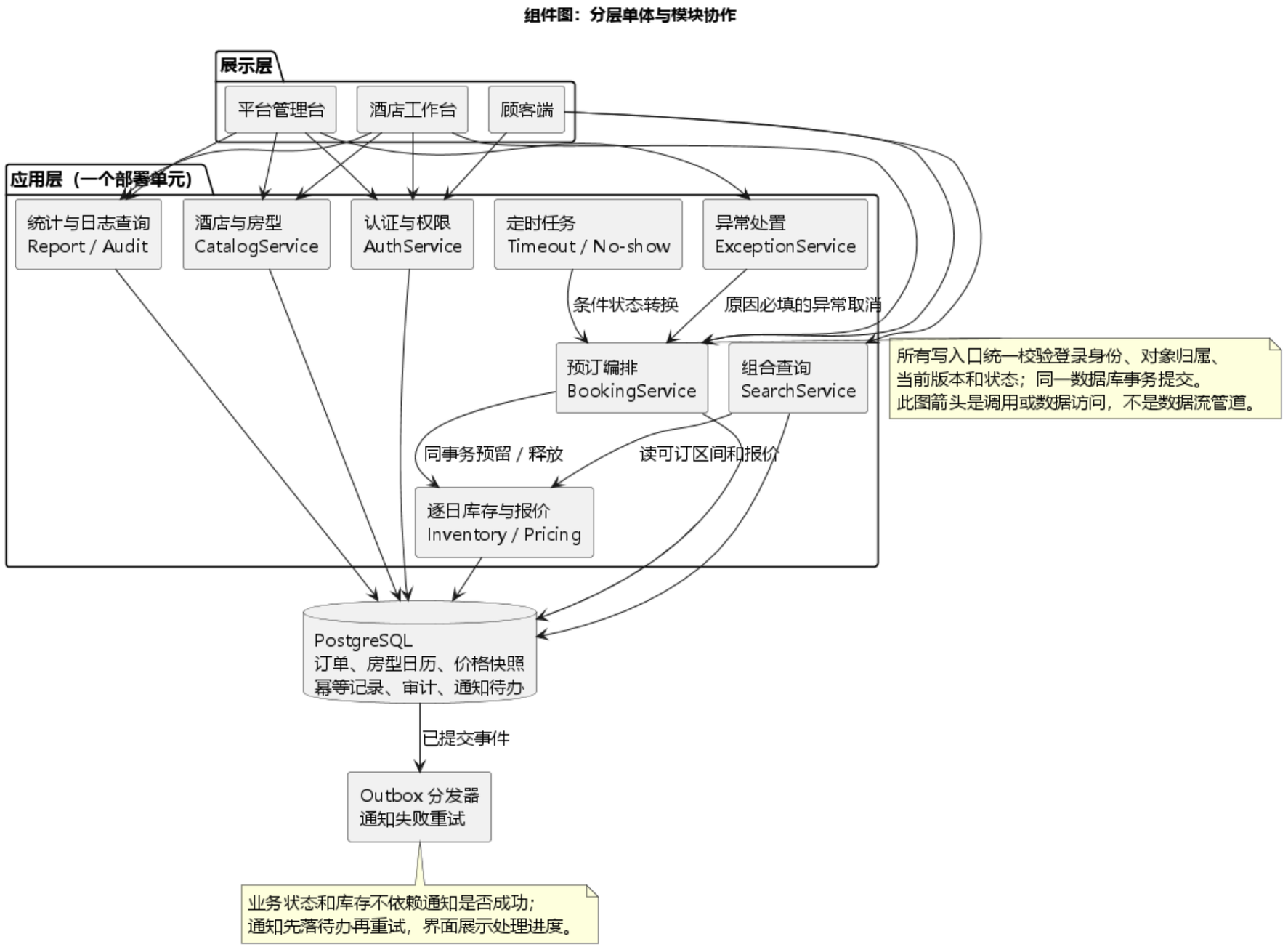


图2 领域类图

结构性图。逻辑关系：一笔订单有多条逐夜记录，每晚关联对应的房型日历。设计原因：连住时需要按天检查房量，并保留下单时的逐夜价格。

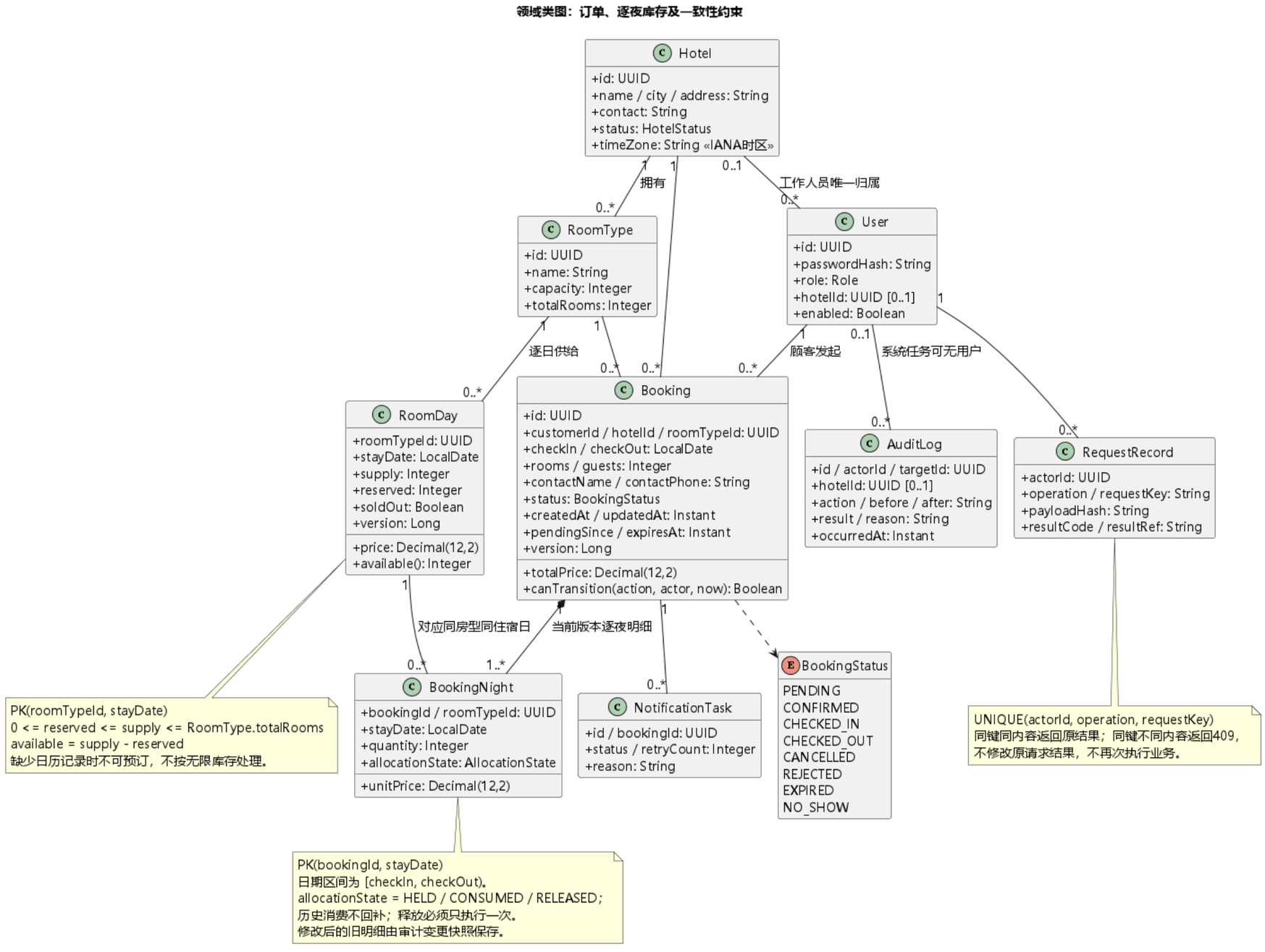


图3 应用服务类图

结构性图。逻辑关系：BookingService调用权限、状态、库存和数据访问接口，定时任务也从这里处理订单。设计原因：不同入口共用一套业务检查。

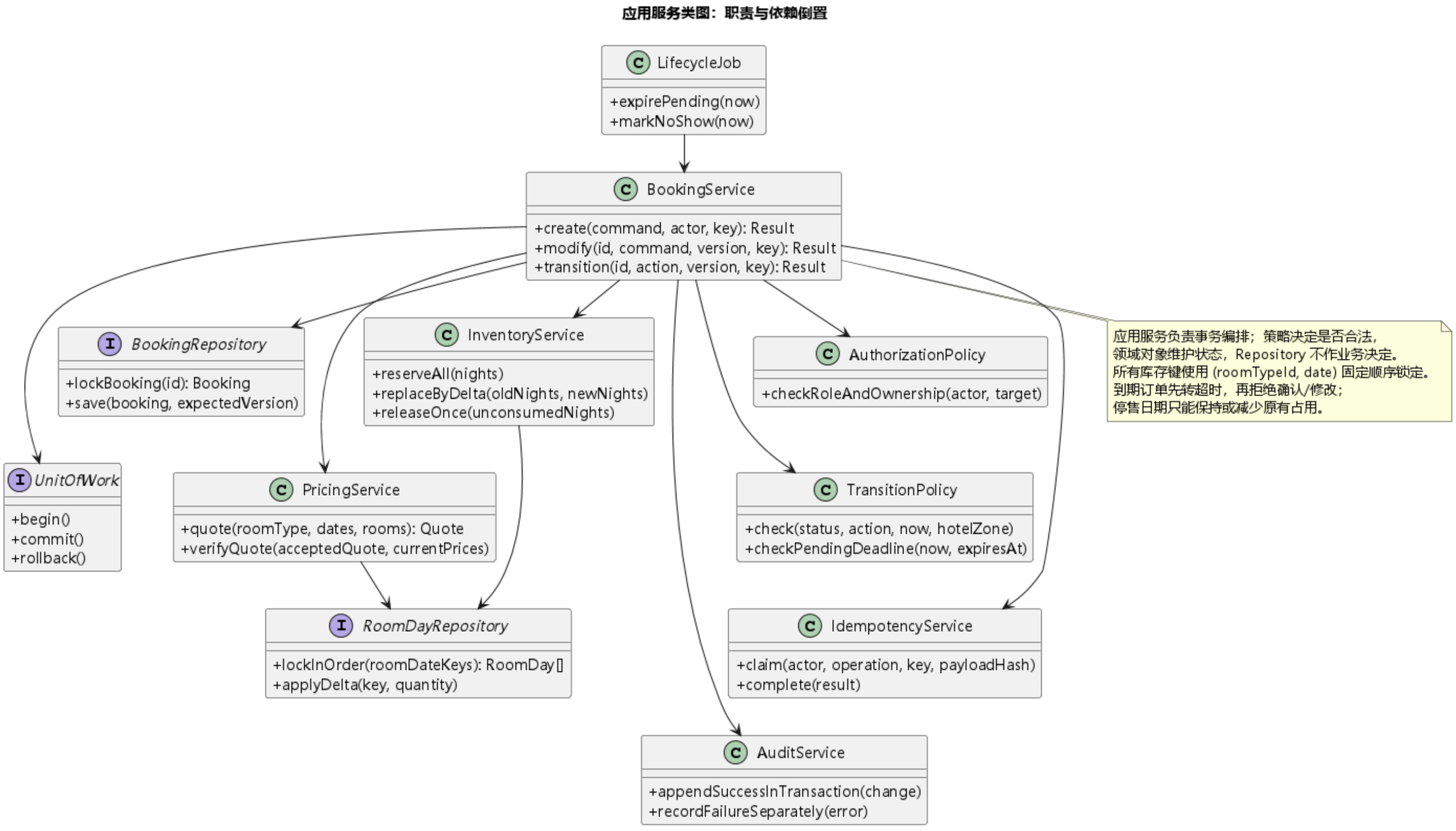


图4 用例图

行为性图。逻辑关系：游客、顾客、酒店人员和管理员分别关联自己的功能。设计原因：限制数据和操作范围，顾客只管理本人订单，酒店人员只处理本酒店业务。

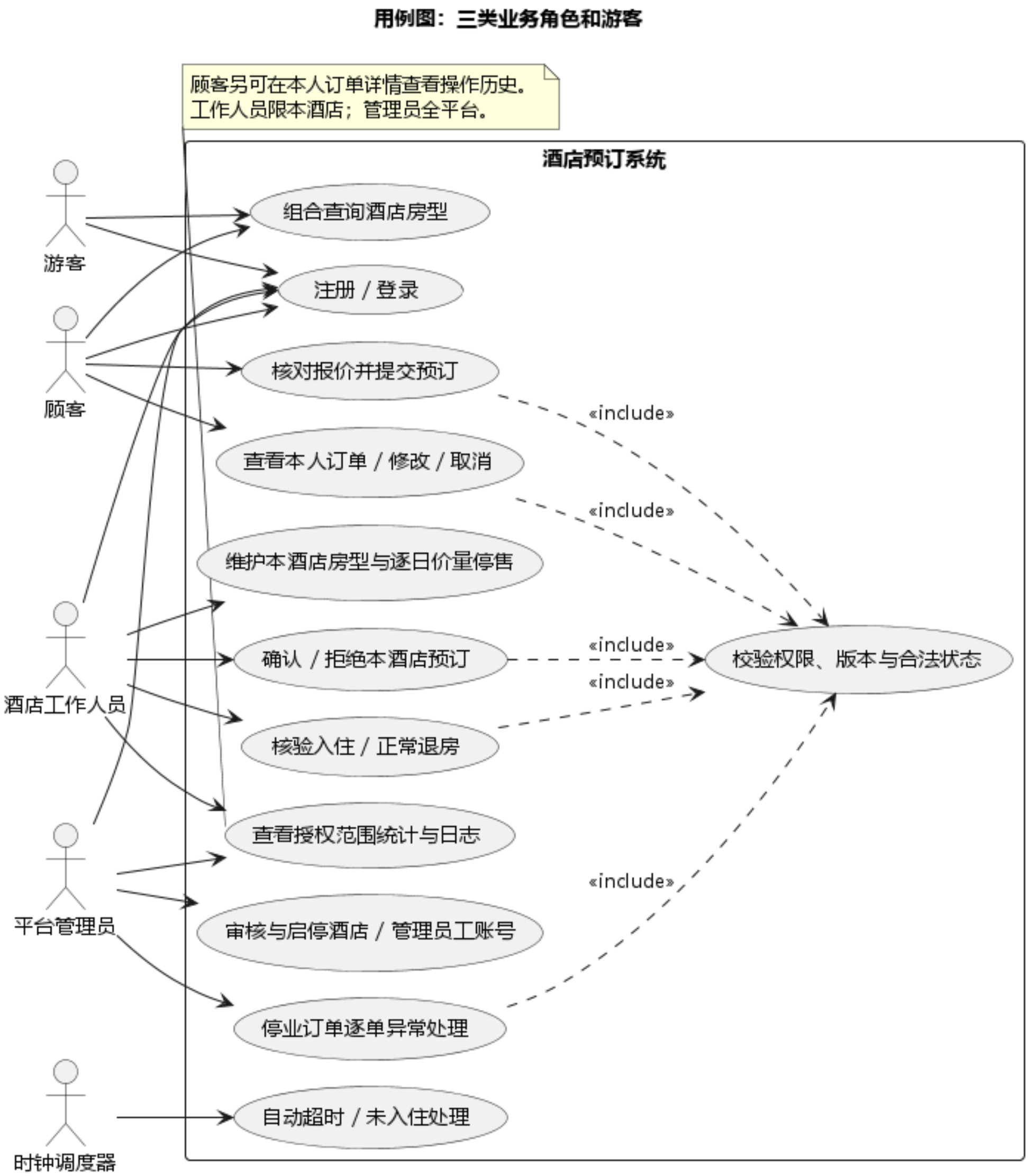


图5 订单状态图

行为性图。逻辑关系：订单按确认、入住、退房及异常处理进入不同状态。设计原因：每次操作要有明确前提，已取消等终态不能再次入住。

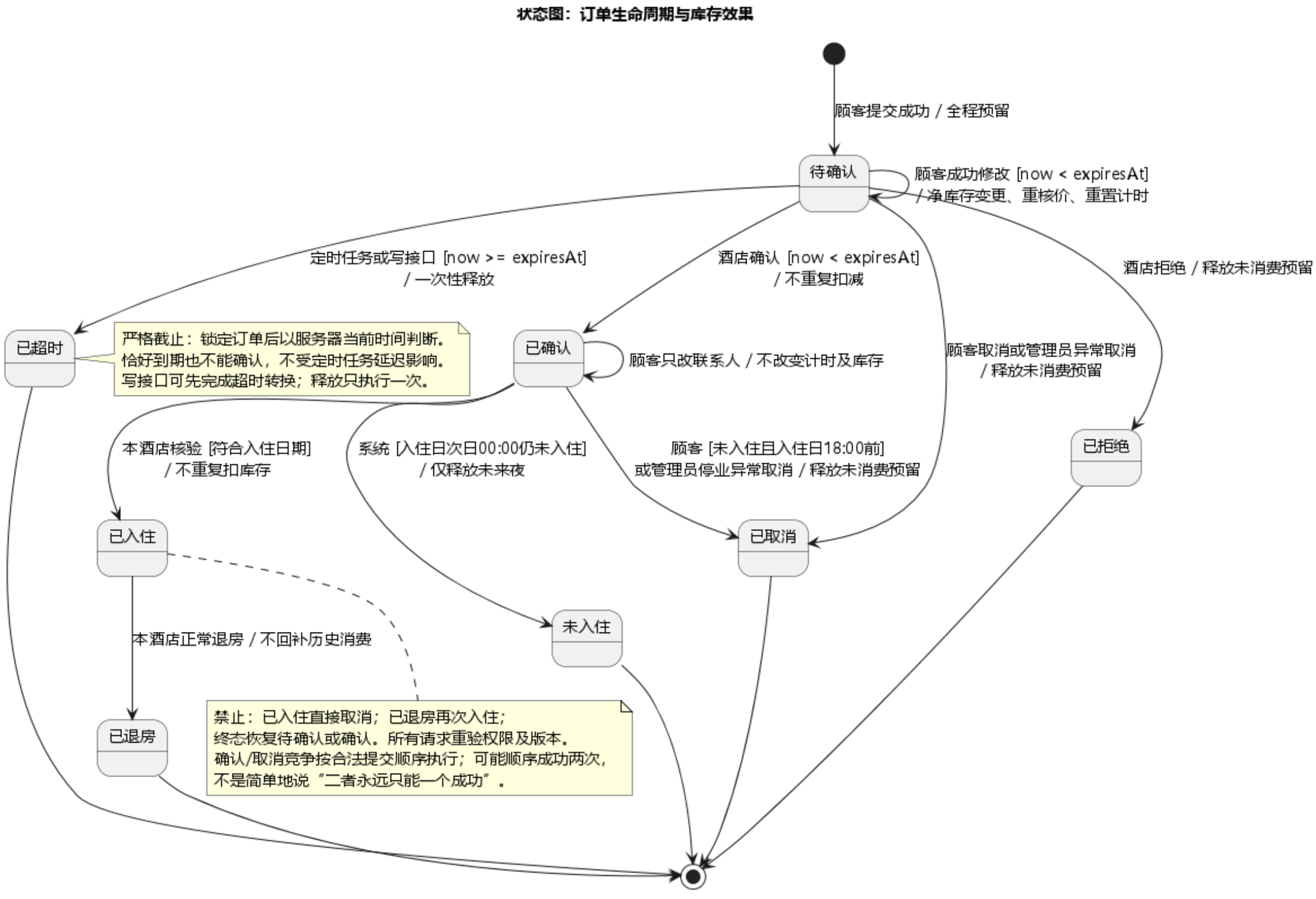


图6 预订活动图

行为性图。流程从输入查询条件到提交结果，包含无房、改价和失败分支。这样可以同时说明正常预订和出错后如何返回提示、撤销修改。

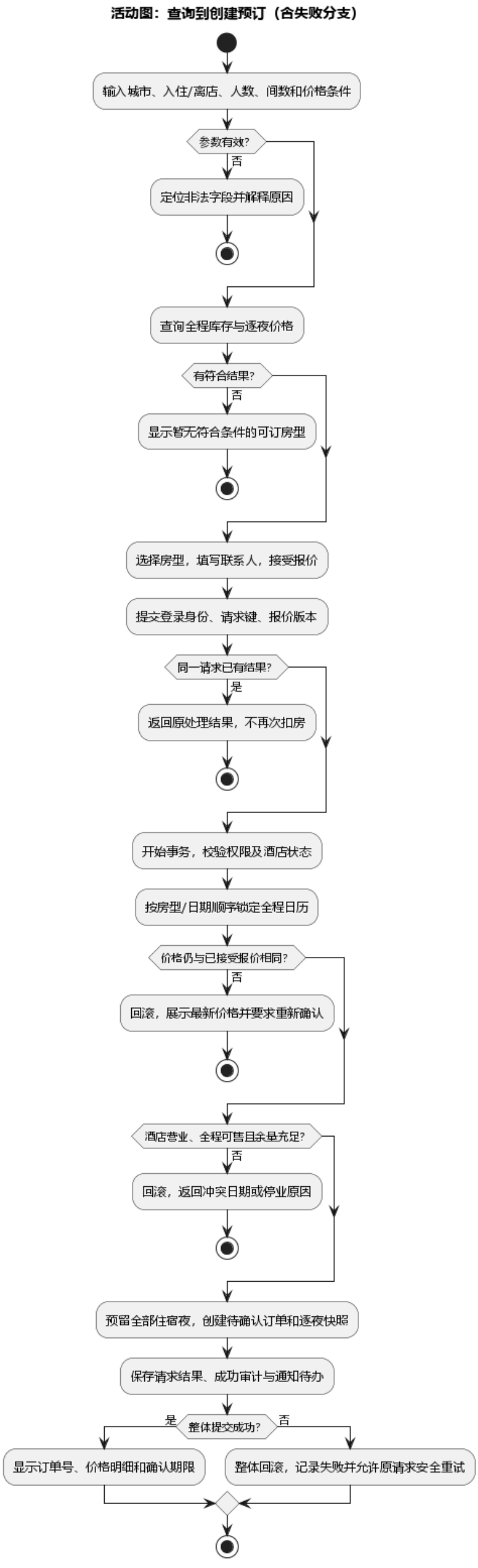


图7 创建预订时序图

交互性图。调用关系：页面提交后，服务检查请求、价格和库存，再提交订单及各晚预留。设计原因：防止重复建单和只占用部分住宿夜。

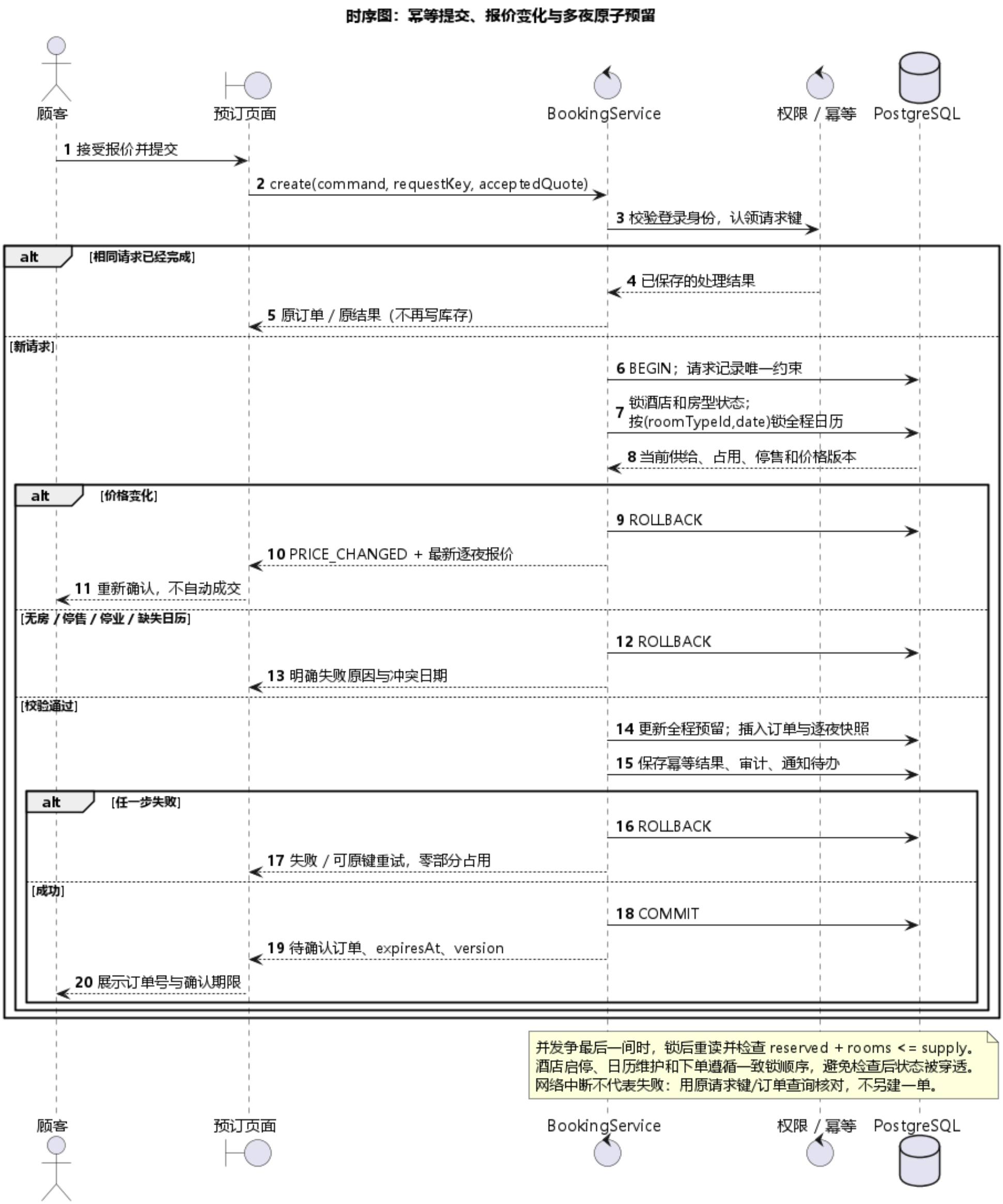


图8 修改预订时序图

交互性图。调用关系：锁定原订单，检查新旧日期并更新占用。设计原因：保留重叠夜原有房量，新日期无房时不影响原订单。

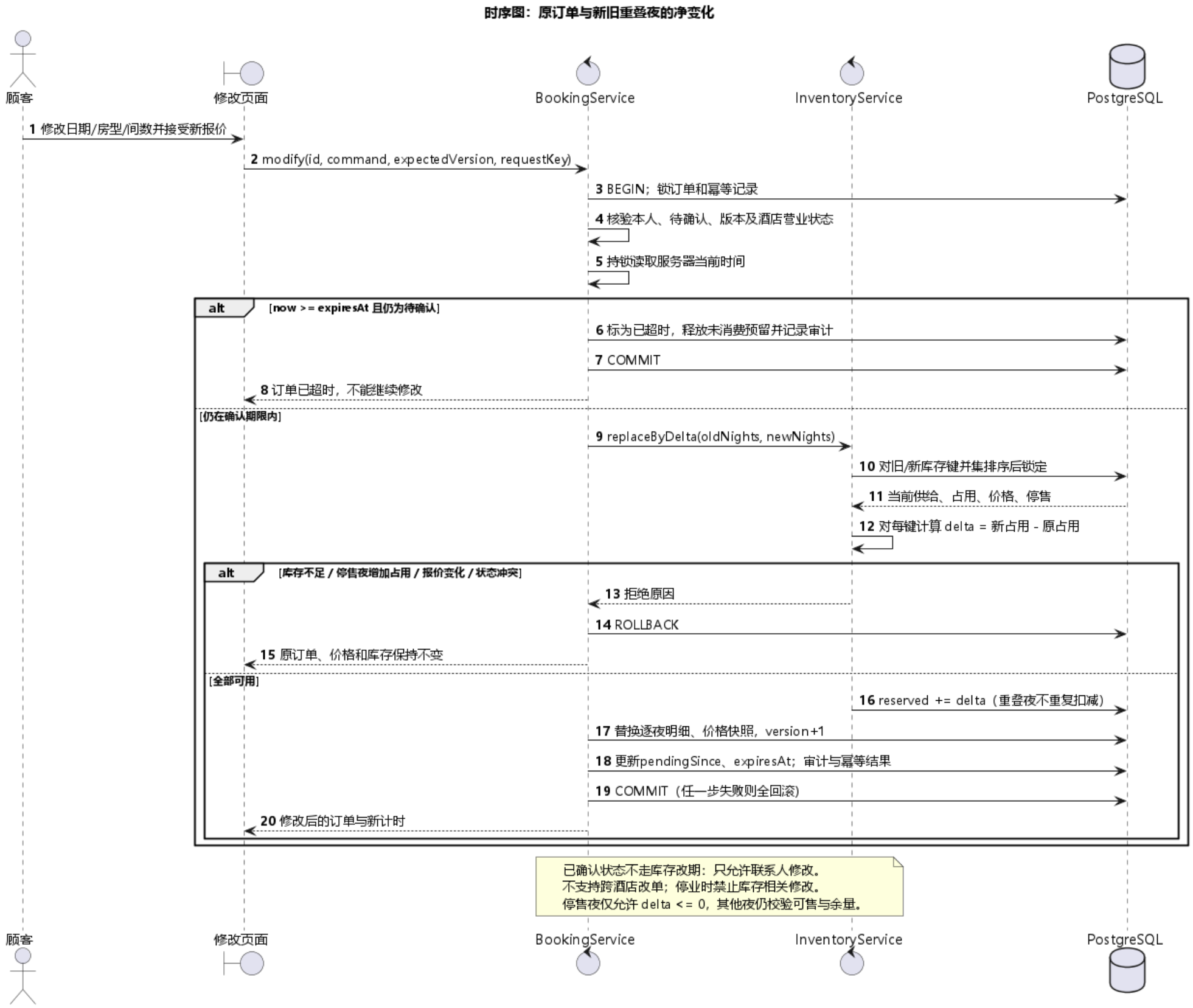
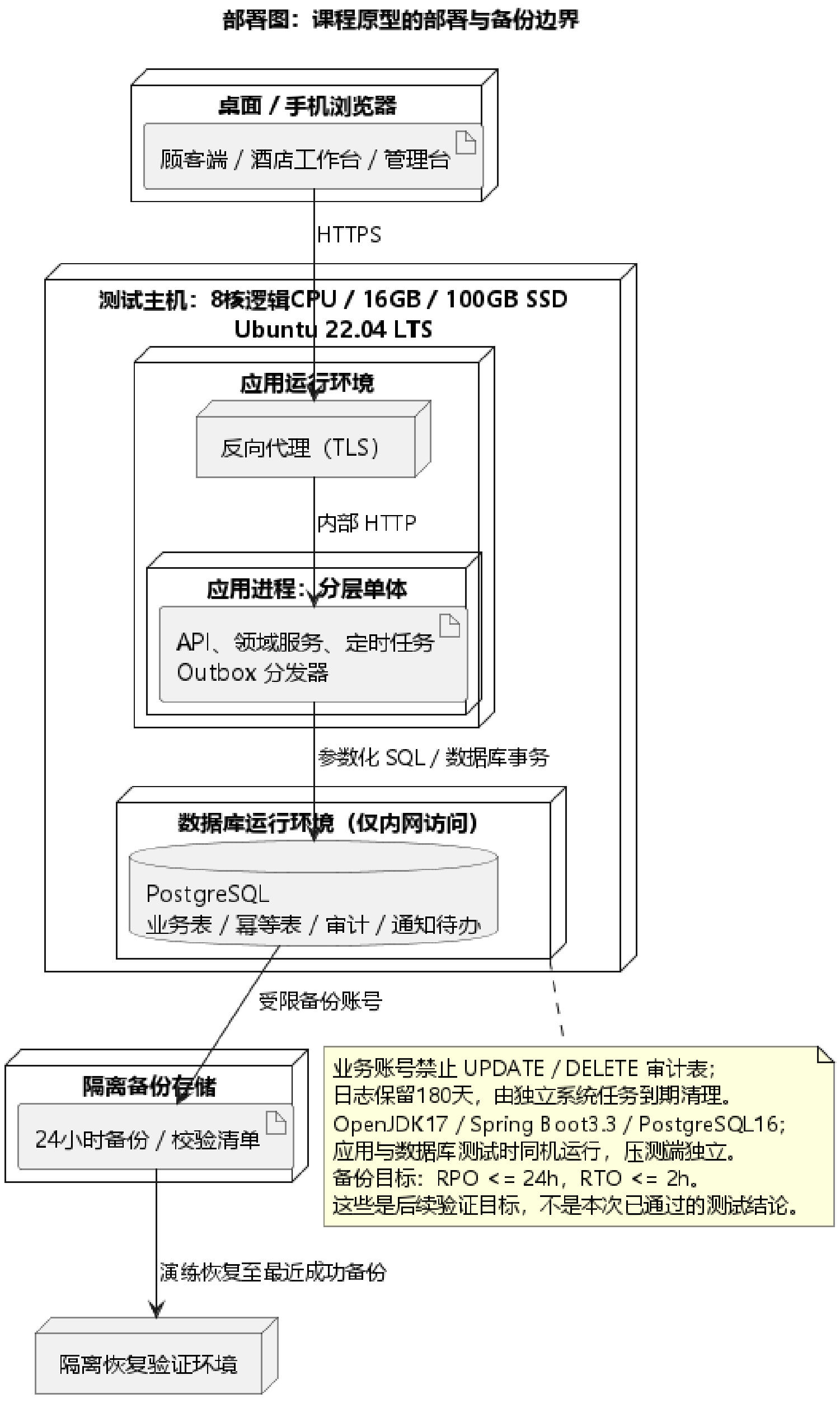


图9 部署图

结构性图。测试时应用与数据库同机运行，压测端独立，备份使用隔离存储。这样可以固定测试条件，恢复演练也不会覆盖正在使用的数据。



UI-01 账号登录与注册

顾客端，对应REQ-001 / 023。顾客自行注册；酒店工作人员账号由平台管理员创建。



UI-02 寻找下一段旅程的落脚处

顾客端，对应REQ-004 / 007。按城市、日期、人数、房型和整段住宿价格组合查询。



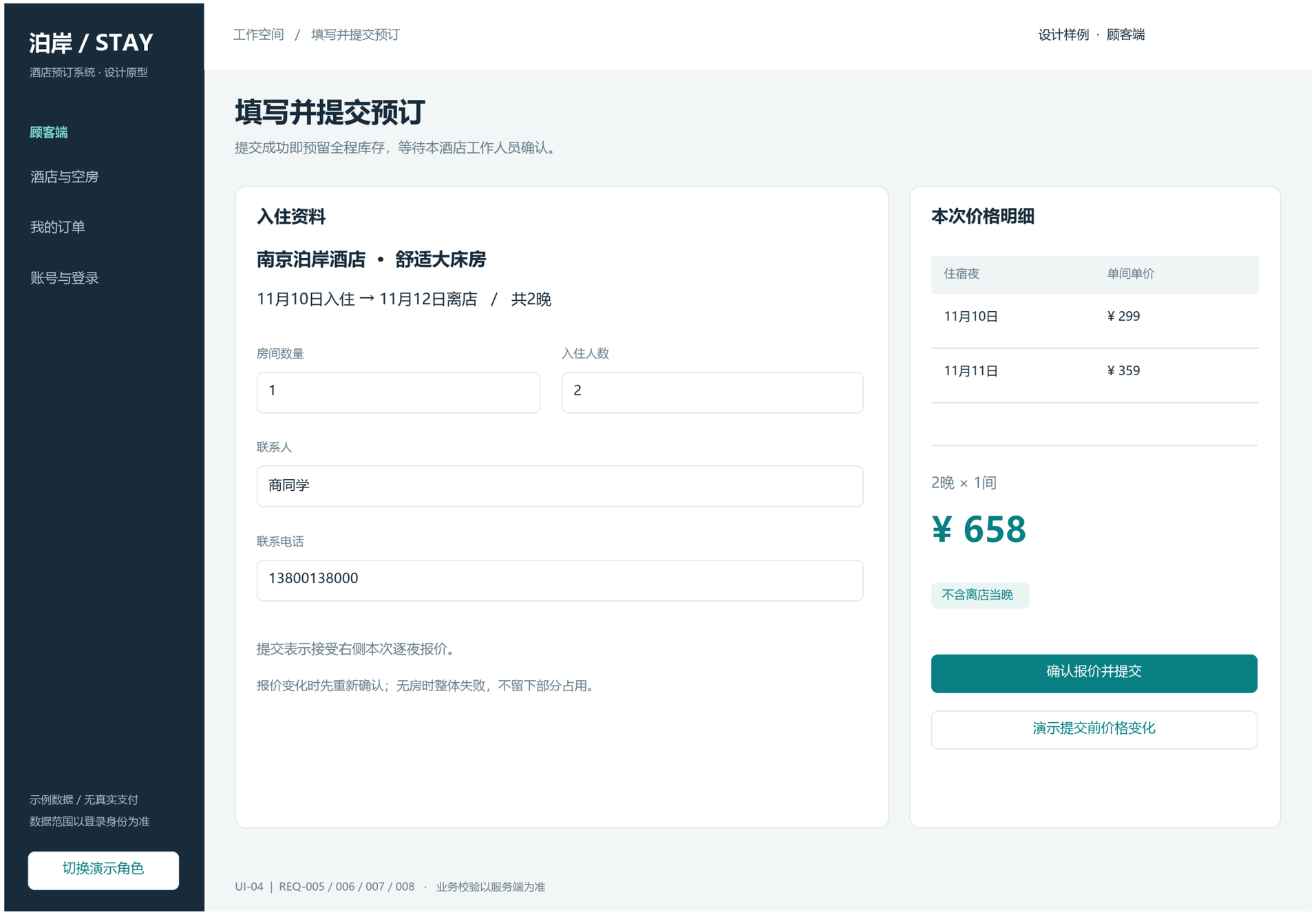
UI-03 南京泊岸酒店

顾客端，对应REQ-002 / 003 / 004。南京市鼓楼区湖南路18号 • 联系电话 025-8000-1234



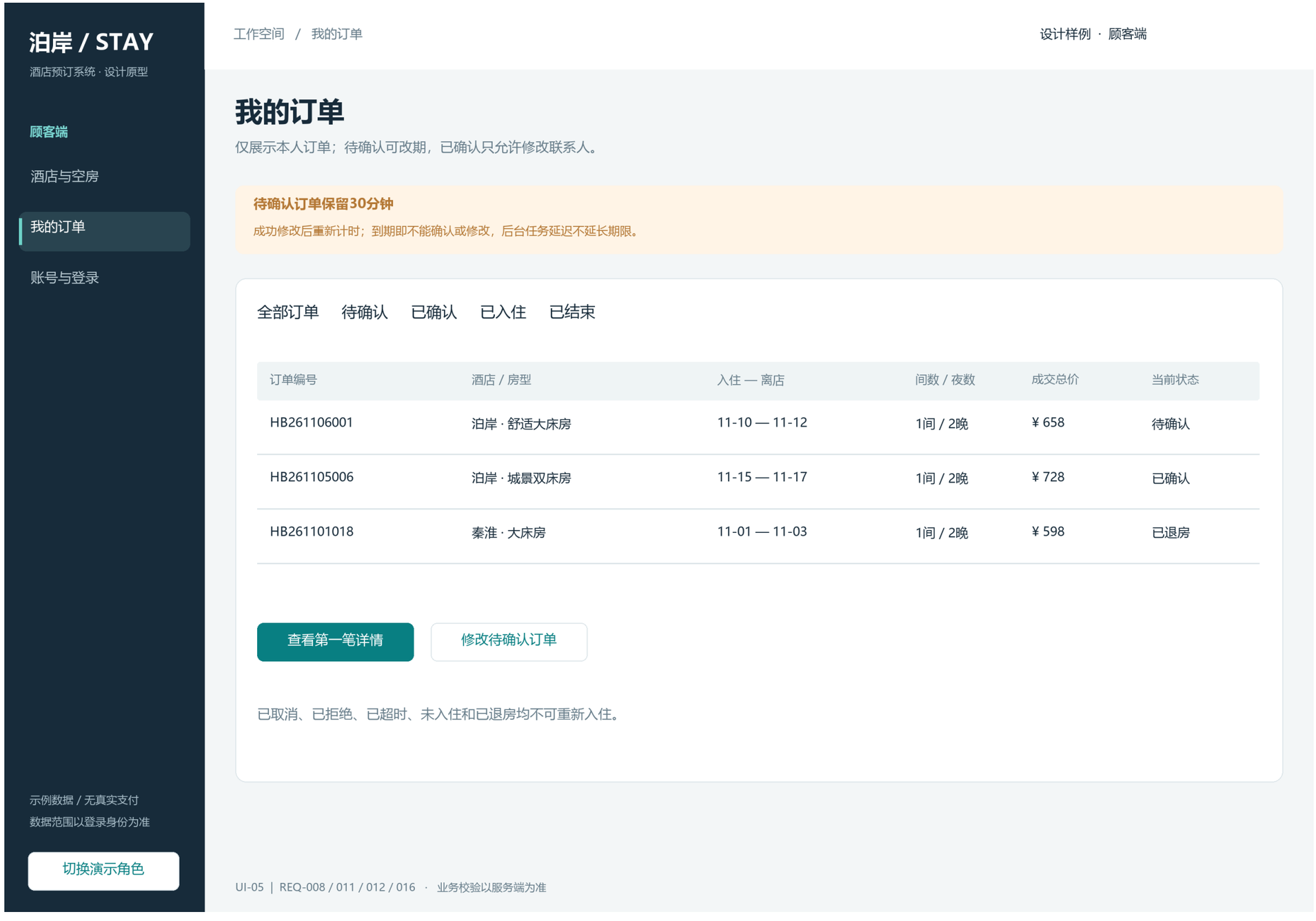
UI-04 填写并提交预订

顾客端，对应REQ-005 / 006 / 007 / 008。提交成功即预留全程库存，等待本酒店工作人员确认。



UI-05 我的订单

顾客端，对应REQ-008 / 011 / 012 / 016。仅展示本人订单；待确认可改期，已确认只允许修改联系人。



UI-06 修改待确认订单

顾客端，对应REQ-006 / 007 / 011。新日期无房时保留原订单；先核价，成功后一次性处理新旧预留。



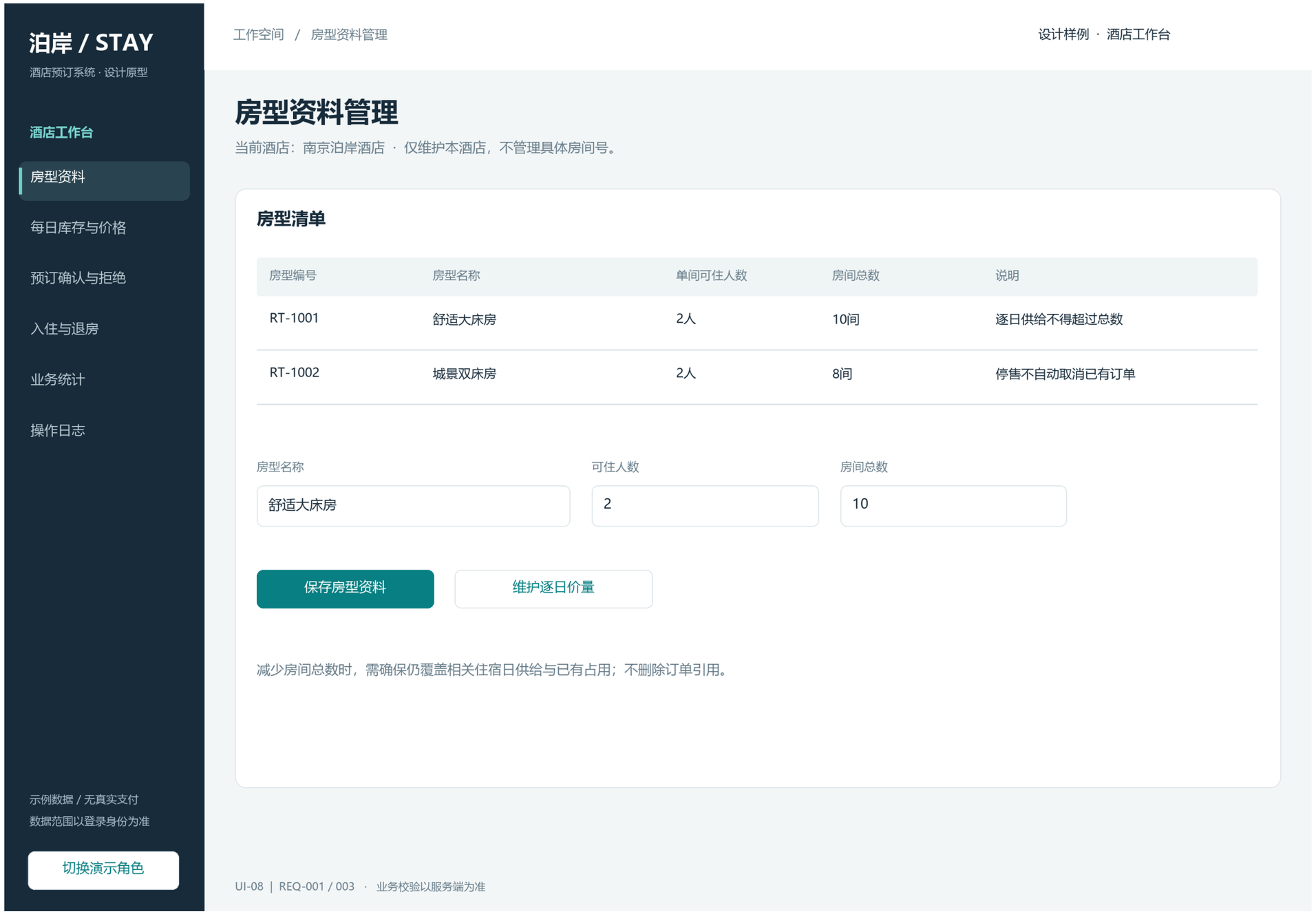
UI-07 订单详情与操作历史

顾客端，对应REQ-008 / 012 / 016 / 019。HB261106001 • 服务端版本 v3 • 当前状态和操作结果以最新数据为准



UI-08 房型资料管理

酒店工作台，对应REQ-001 / 003。当前酒店：南京泊岸酒店 • 仅维护本酒店，不管理具体房间号。



UI-09 每日库存、价格与停售

酒店工作台，对应REQ-003 / 006 / 007 / 017。房型 RT-1001 • 舒适大床房 | 分清供给总量、已占用与剩余量。

泊岸 / STAY

酒店预订系统 · 设计原型

酒店工作台

房型资料

每日库存与价格

预订确认与拒绝

入住与退房

业务统计

操作日志

示例数据 / 无真实支付

数据范围以登录身份为准

切换演示角色

工作空间 / 每日库存、价格与停售

设计样例 · 酒店工作台

每日库存、价格与停售

房型 RT-1001 · 舒适大床房 | 分清供给总量、已占用与剩余量。

剩余房量 = 当日供给总量 − 仍应占用的房量

停售日期不能增加占用，原订单可减少或释放占用；已生成订单的价格快照保持不变。

住宿日期	供给总量	已占用	剩余	单晚价格	销售状态
2026-11-10	3间	2间	1间	¥ 299	可售
2026-11-11	1间	0间	1间	¥ 359	可售
2026-11-12	3间	1间	2间	¥ 329	停售

11月10日供给总量

3

11月10日单晚价格

299

销售状态

可售

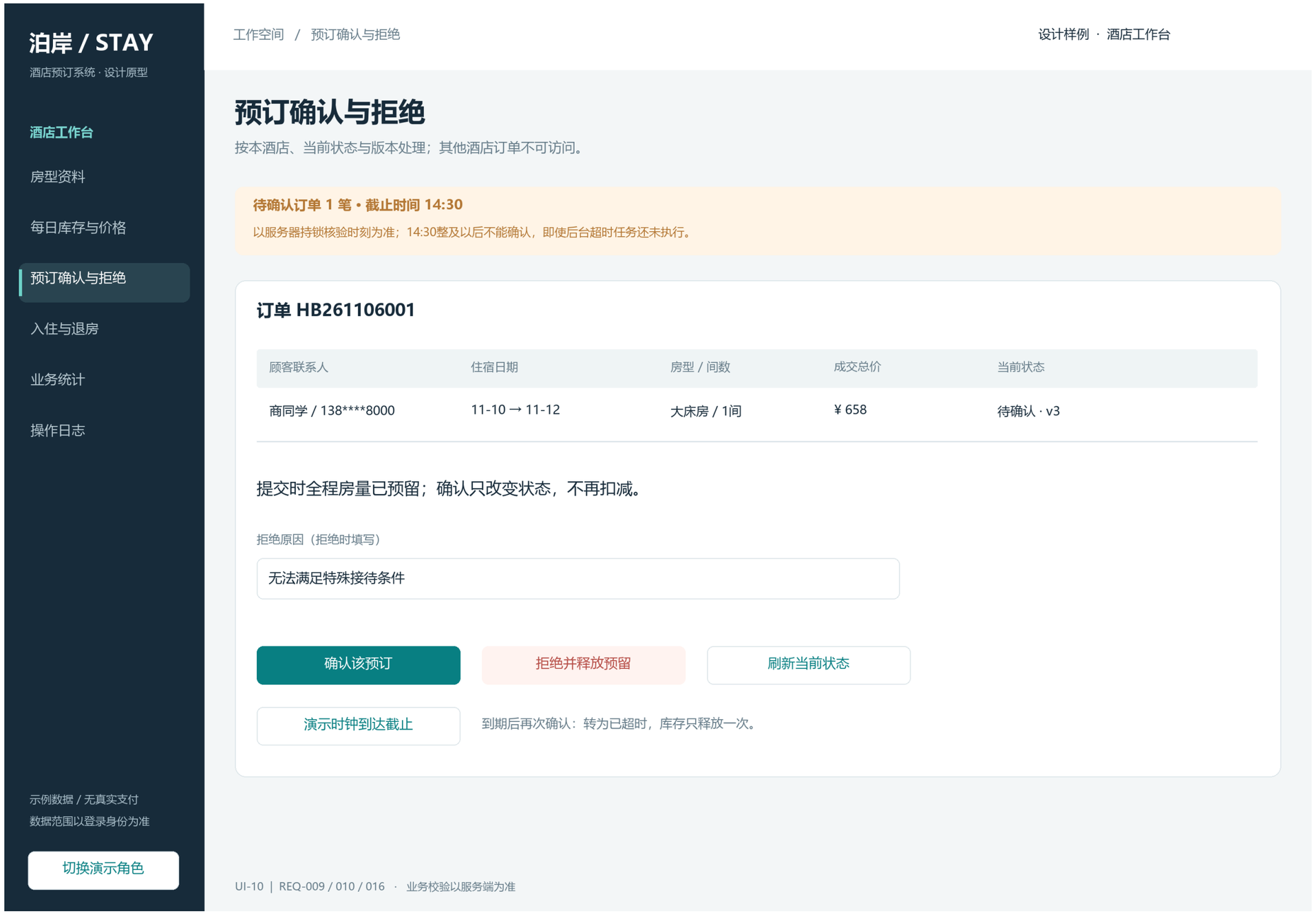
保存当日变更

示例：总量3、占用2，还能再售1间；若把总量调成1，必须拒绝，不能负库存。

UI-09 | REQ-003 / 006 / 007 / 017 · 业务校验以服务端为准

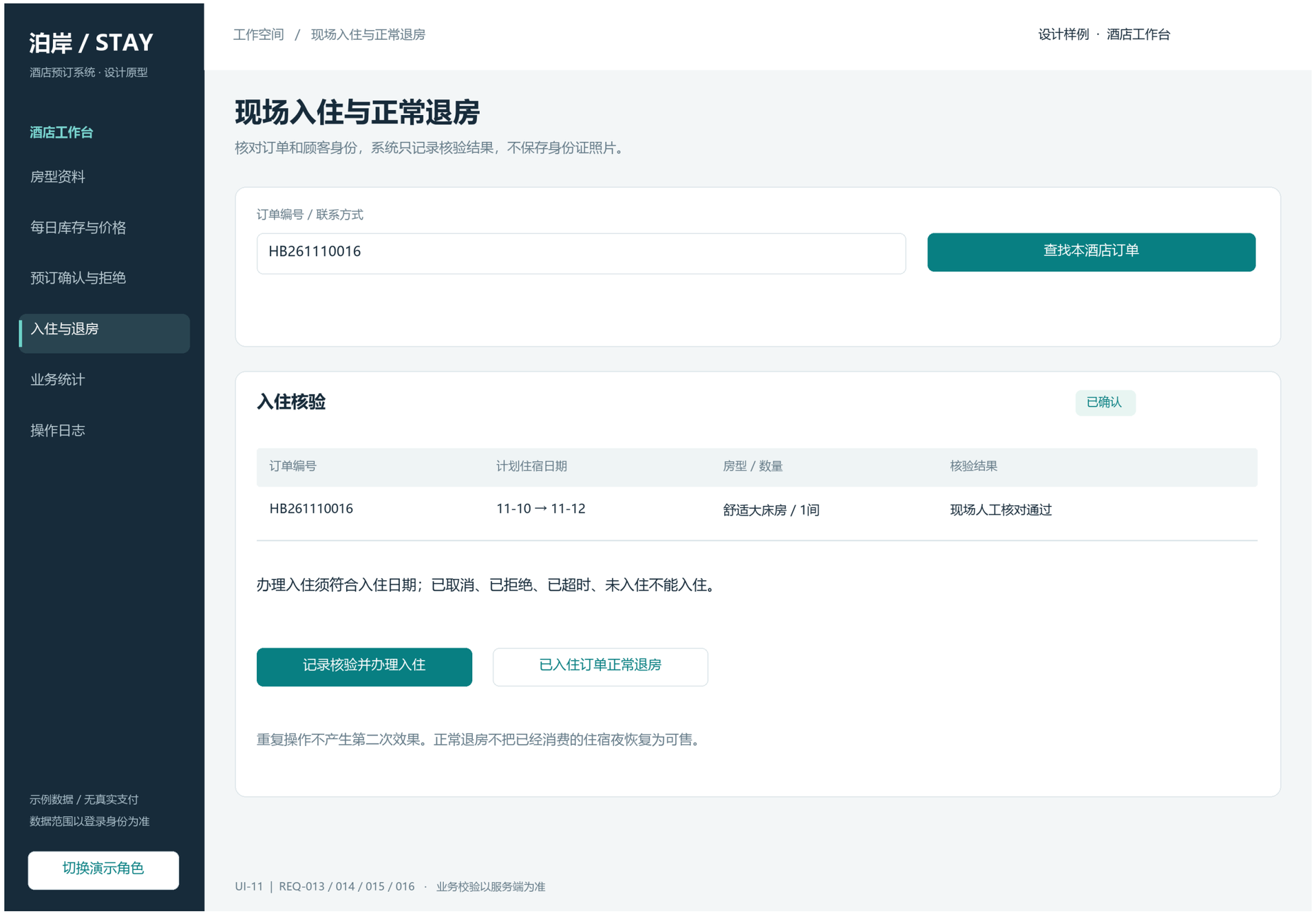
UI-10 预订确认与拒绝

酒店工作台，对应REQ-009 / 010 / 016。按本酒店、当前状态与版本处理；其他酒店订单不可访问。



UI-11 现场入住与正常退房

酒店工作台，对应REQ-013 / 014 / 015 / 016。核对订单和顾客身份，系统只记录核验结果，不保存身份证照片。



UI-12 酒店审核、启停与异常处置

平台管理台，对应REQ-002 / 017 / 019。停业阻止新预订；已有订单分类处理，不能抹去历史记录。



UI-13 工作人员账号管理

平台管理台，对应REQ-001 / 020 / 023。每名工作人员绑定唯一酒店；停用账号不删除历史操作记录。



UI-14 预订数量与间夜统计

酒店工作台，对应REQ-018 / 001。工作人员限本酒店，管理员可切换全平台；不统计收入或入住率。



UI-15 操作日志与追踪

酒店工作台，对应REQ-019 / 023 / 024。日志只读、保留180天；管理员查全平台，酒店人员只查本酒店。



UI-16 移动端订单详情

顾客端，对应REQ-025。手机关键任务适配

